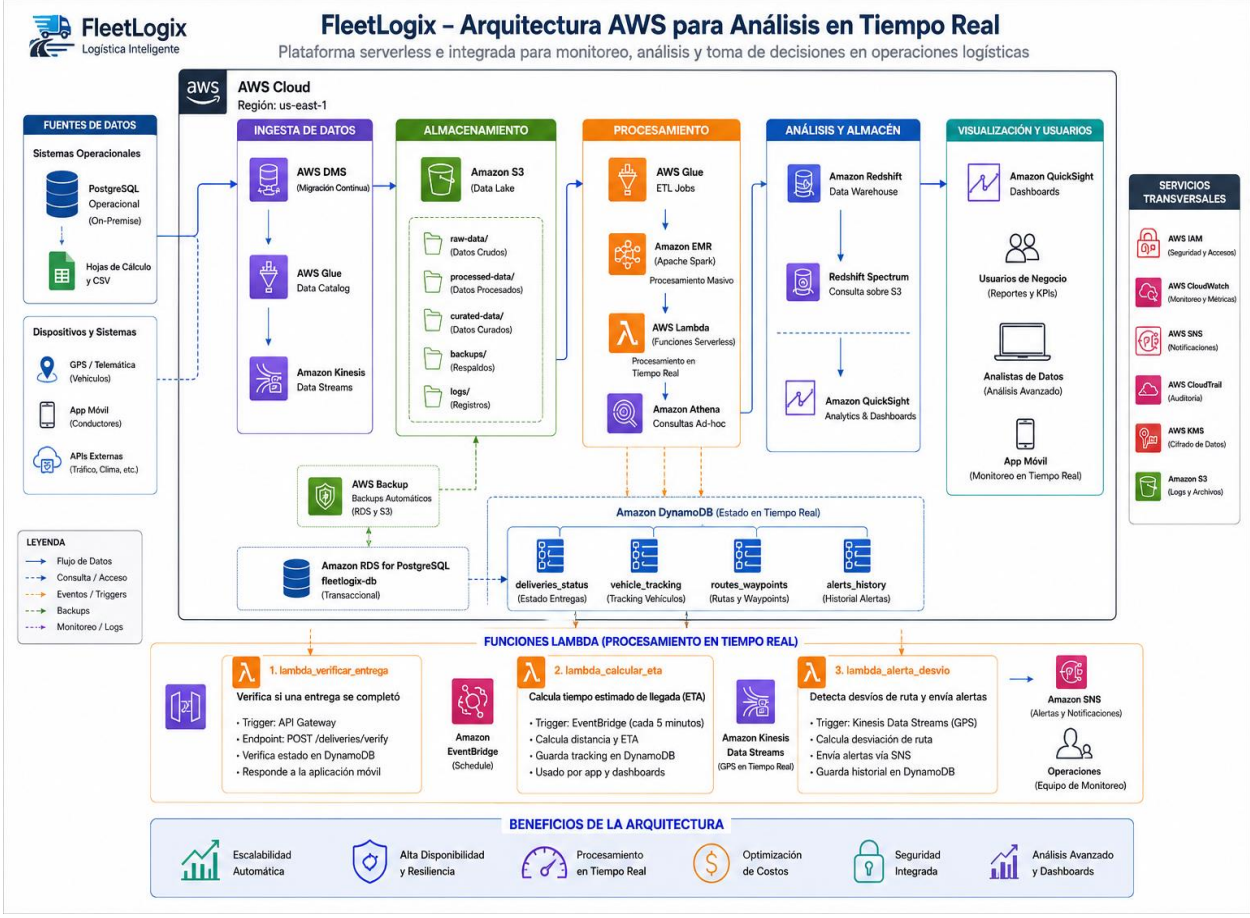


Arquitectura AWS

La arquitectura de FleetLogix fue diseñada para integrar el procesamiento de datos históricos y el monitoreo en tiempo real dentro de un entorno escalable y seguro en AWS. El objetivo principal es centralizar la información proveniente de diferentes fuentes, transformarla mediante procesos ETL y ponerla a disposición de los usuarios de negocio a través de dashboards e indicadores que faciliten la toma de decisiones.

El flujo de datos inicia con las fuentes operacionales, donde PostgreSQL almacena la información transaccional generada por la operación logística, incluyendo vehículos, conductores, rutas, viajes, entregas y mantenimientos. Adicionalmente, la plataforma puede recibir información desde archivos CSV, hojas de cálculo, dispositivos GPS instalados en los vehículos, aplicaciones móviles utilizadas por los conductores y APIs externas que proporcionan información como tráfico, clima y mapas.

Para la ingesta de datos, AWS Database Migration Service (DMS) replica continuamente la información almacenada en PostgreSQL hacia la nube, permitiendo mantener sincronizados los datos sin afectar el rendimiento del sistema transaccional. Paralelamente, AWS Glue Data Catalog registra la estructura y el esquema de los datos, facilitando su descubrimiento y utilización por parte de los diferentes servicios analíticos. Cuando se generan eventos en tiempo real, como actualizaciones de ubicación GPS o cambios en el estado de una entrega, Amazon Kinesis recibe estos flujos de información para su procesamiento inmediato.

Toda la información es almacenada inicialmente en Amazon S3, el cual funciona como el Data Lake de la solución. Dentro de este repositorio se organizan diferentes zonas según el estado de los datos: una zona *Raw* donde se conservan los datos originales, una zona *Processed* con información previamente transformada y validada, una zona *Curated* con datos optimizados para análisis, además de áreas destinadas al almacenamiento de respaldos y registros de ejecución del sistema.

El procesamiento de los datos es realizado principalmente por AWS Glue, donde se ejecutan los procesos ETL encargados de extraer la información, limpiarla, validar reglas de negocio, calcular indicadores y preparar los datos para su análisis. Cuando el volumen de información aumenta considerablemente, Amazon EMR utiliza Apache Spark para distribuir las cargas de trabajo y procesar grandes cantidades de registros de forma paralela. Para consultas rápidas directamente sobre los archivos almacenados en S3, Amazon Athena permite ejecutar consultas SQL sin necesidad de cargar previamente los datos en una base de datos.

Una vez transformada, la información es cargada en Amazon Redshift, que actúa como el Data Warehouse de la organización. Allí se implementa un modelo dimensional compuesto por tablas de dimensiones y tablas de hechos, optimizado para consultas analíticas de alto rendimiento. En aquellos casos donde se requiere consultar directamente archivos almacenados en el Data Lake, Redshift Spectrum permite acceder a la información sin necesidad de duplicarla dentro del Data Warehouse.

Los datos consolidados son consumidos por Amazon QuickSight, donde se construyen dashboards interactivos e indicadores de desempeño relacionados con entregas, consumo de combustible, eficiencia operativa, desempeño de conductores, tiempos de entrega y demás métricas estratégicas. Estos tableros son utilizados tanto por usuarios de negocio como por analistas de datos para apoyar la toma de decisiones basadas en información actualizada.

Además del procesamiento histórico, la arquitectura incorpora un componente de procesamiento en tiempo real. La información proveniente de dispositivos GPS y aplicaciones móviles es almacenada temporalmente en Amazon DynamoDB, donde se mantiene el estado actual de las entregas, la ubicación de los vehículos, los puntos de las rutas y el historial de alertas generadas durante la operación.

Sobre esta información actúan varias funciones implementadas con AWS Lambda. Estas funciones ejecutan procesos automáticos como la verificación del estado de una entrega, el cálculo periódico del tiempo estimado de llegada (ETA) y la detección de desvíos de ruta mediante el análisis continuo de la información recibida desde Amazon Kinesis. Cuando se identifica un evento importante, las funciones generan notificaciones utilizando Amazon SNS, permitiendo alertar oportunamente a los equipos de operación.

Toda la plataforma se encuentra respaldada por servicios transversales que garantizan la seguridad, disponibilidad y monitoreo de la solución. AWS IAM administra la autenticación y los permisos de acceso a los diferentes recursos, AWS CloudWatch supervisa el rendimiento del sistema y registra métricas y eventos, AWS CloudTrail mantiene un historial de auditoría de todas las acciones realizadas dentro de la infraestructura, AWS KMS protege la información mediante cifrado y AWS Backup automatiza la generación de copias de seguridad de los servicios críticos.

Gracias a esta arquitectura, FleetLogix dispone de una plataforma moderna, escalable y altamente disponible que integra el procesamiento de datos históricos con análisis en tiempo real, permitiendo optimizar la operación logística, automatizar procesos críticos y ofrecer información confiable para la toma de decisiones estratégicas.

AWS Analisis Arquitectura

Introducción

Este documento describe el diseño de la arquitectura AWS implementada para FleetLogix, explicando los servicios utilizados, el proceso de configuración de la infraestructura, el funcionamiento de las funciones Lambda y el procedimiento de migración de la base de datos PostgreSQL hacia Amazon RDS. El objetivo es documentar el comportamiento del sistema sin necesidad de ejecutarlo, permitiendo comprender cómo interactúan los diferentes componentes dentro de la solución propuesta.

Recursos creados en aws_setup.py

El archivo aws_setup.py automatiza la creación y configuración de la infraestructura básica de AWS necesaria para soportar la plataforma FleetLogix. Su propósito es preparar el entorno donde posteriormente se ejecutarán los procesos ETL, las funciones Lambda y el almacenamiento de información histórica y en tiempo real.

Amazon RDS PostgreSQL

El script crea una instancia administrada de PostgreSQL utilizando Amazon RDS.
La configuración establece una instancia db.t3.micro, correspondiente al nivel gratuito de AWS, con PostgreSQL versión 15.4, 20 GB de almacenamiento, respaldos automáticos durante siete días y ventanas definidas para mantenimiento y copias de seguridad.
El propósito de esta instancia es alojar la base de datos operacional de FleetLogix dentro de la nube, eliminando la necesidad de administrar un servidor propio y garantizando alta disponibilidad y recuperación ante fallos.

Amazon S3

Posteriormente se crea un bucket denominado fleetlogix-data, utilizado como Data Lake del proyecto.

Dentro del bucket se crean automáticamente las carpetas:

- raw-data/
- processed-data/
- backups/
- logs/

Además, se configura una regla de ciclo de vida (*Lifecycle Policy*) que mueve automáticamente los datos almacenados en la carpeta raw-data hacia Glacier después de 90 días, reduciendo significativamente los costos de almacenamiento.

Amazon DynamoDB

El script crea cuatro tablas NoSQL destinadas al procesamiento en tiempo real.

deliveries_status

Almacena el estado actual de cada entrega, permitiendo conocer si una orden se encuentra pendiente, en tránsito o entregada.

vehicle_tracking

Registra continuamente la ubicación GPS de cada vehículo junto con la fecha y hora de la actualización.

routes_waypoints

Almacena los puntos GPS que conforman cada ruta planificada, información utilizada posteriormente para detectar desviaciones.

alerts_history

Conserva el historial completo de alertas generadas por el sistema durante la operación logística.

AWS Backup

Después de crear la instancia RDS, el script genera un snapshot inicial y configura una política de retención de siete días para los respaldos automáticos.

Esto garantiza la recuperación de la información ante posibles fallos del sistema.

AWS IAM

Finalmente se crea un rol denominado FleetLogixLambdaRole, el cual será utilizado por todas las funciones Lambda.

A este rol se le asignan las políticas necesarias para acceder a:

- AWS Lambda Basic Execution
- Amazon DynamoDB
- Amazon S3
- Amazon SNS

De esta manera las funciones Lambda pueden consultar tablas, almacenar información y enviar notificaciones sin utilizar credenciales explícitas.

Funciones implementadas en lambda_functions.py

El archivo lambda_functions.py contiene tres funciones Lambda diseñadas para automatizar procesos críticos de la operación logística mediante un modelo completamente serverless.

Cada función responde a un evento diferente y ejecuta tareas específicas dentro de la plataforma.

Función Lambda: Verificar Entrega

La función lambda_verificar_entrega() tiene como objetivo verificar si una entrega ya fue completada consultando directamente la tabla deliveries_status almacenada en Amazon DynamoDB.

Cuando la aplicación móvil registra una entrega, envía un evento que contiene el identificador de la entrega (delivery_id) y, opcionalmente, el número de seguimiento (tracking_number).

La función valida inicialmente que el identificador haya sido recibido. Si este dato no existe, responde inmediatamente con un código HTTP 400 indicando que el parámetro es obligatorio.

Posteriormente establece conexión con la tabla deliveries_status y busca el registro correspondiente mediante la clave primaria delivery_id.

Si el registro existe, recupera el estado actual de la entrega, verifica si dicho estado corresponde a delivered y construye una respuesta que incluye el identificador de la entrega, el número de seguimiento, el estado actual y la fecha de entrega registrada.

Cuando la entrega no existe en DynamoDB, la función devuelve un código HTTP 404 indicando que el recurso solicitado no fue encontrado.

En caso de producirse cualquier excepción durante la consulta, la función captura el error y devuelve un código HTTP 500 junto con el mensaje correspondiente.

Este proceso permite que la aplicación móvil consulte en tiempo real el estado actualizado de cualquier entrega sin necesidad de acceder directamente a la base de datos principal.

Función Lambda: Calcular ETA

La función lambda_calcular_eta() calcula el tiempo estimado de llegada (ETA) de un vehículo utilizando la ubicación actual, el destino y la velocidad reportada.

Para ello recibe como parámetros el identificador del vehículo, la ubicación GPS actual, las coordenadas del destino y la velocidad promedio.

La distancia restante se calcula mediante una aproximación simplificada basada en coordenadas geográficas. Posteriormente se estima el tiempo restante utilizando la velocidad del vehículo.

Finalmente, la información calculada se almacena en la tabla vehicle_tracking, permitiendo que aplicaciones móviles y dashboards consulten permanentemente el progreso de cada vehículo.

Función Lambda: Alerta de Desvío

La función lambda_alerta_desvio() supervisa continuamente la ubicación GPS de los vehículos.

Cada vez que recibe una nueva posición consulta en DynamoDB los waypoints correspondientes a la ruta asignada.

Posteriormente calcula la distancia mínima entre la posición actual del vehículo y la ruta planificada.

Cuando dicha distancia supera el umbral establecido de cinco kilómetros, el sistema considera que existe un desvío de ruta.

En ese momento se genera un mensaje con la información del incidente, el cual es enviado mediante Amazon SNS para notificar al equipo de operaciones.

Adicionalmente la alerta queda registrada en la tabla alerts_history, permitiendo conservar un historial completo de todos los eventos detectados.

Explicación de la función migrar_datos_postgresql()

La función migrar_datos_postgresql() no realiza directamente la migración de la base de datos. Su propósito consiste en generar automáticamente un script Bash denominado migrate_to_rds.sh, el cual contiene todas las instrucciones necesarias para copiar la información desde PostgreSQL local hacia Amazon RDS.

El proceso comienza realizando una copia completa de la base de datos local mediante el comando pg_dump, generando un archivo SQL que contiene tanto la estructura como los datos de todas las tablas.

Posteriormente se establece conexión con la instancia PostgreSQL alojada en Amazon RDS y se crea la base de datos destino.

Finalmente, el archivo generado por pg_dump es restaurado utilizando psql, recreando la estructura original e importando toda la información dentro de la instancia administrada por AWS.

Este procedimiento permite migrar completamente la base de datos operacional hacia la nube sin modificar la estructura existente, facilitando la continuidad de los procesos ETL y del entorno analítico implementado en FleetLogix.